www.tntextbooks.in

11.6.2 Call by reference or address Method

This method copies the address of the actual argument into the formal parameter. Since the address of the argument is passed ,any change made in the formal parameter will be reflected back in the actual parameter.

Program 11.18

```
#include<iostream>
using namespace std;
void display(int &x) //passing address of a//
{
        x=x*x;
        cout<<"\n\nThe Value inside display function (n1 x n1) :"<<x ;
 }
int main()
{
int n1;
cout<<"\nEnter the Value for N1 :";
cin>>n1;
cout<<"\nThe Value of N1 is inside main function Before passing : "<< n1;
display(n1);
cout<<"\nThe Value of N1 is inside main function After passing (n1 x n1)  : "<< n1; return(0);
}
```

Output :

```
Enter the Value for N1 :45
The Value of N1 is inside main function Before passing : 45
The Value inside display function (n1 x n1) :2025
The Value of N1 is inside main function After passing (n1 x n1)  : 2025
```

Note that the only change in the **display()** function is in the function header. The **&** symbol in the declaration of the parameter **x** means that the argument is a reference variable and hence the function will be called by passing reference. Hence when the argument **num1** is passed to the **display()** function, the variable **x** gets the address of **num1** so that the location will be shared. In other words, the variables **x** and **num1** refer to the same memory location. We use the name **num1** in the main() function, and the name **x** in the **display()** function to refer the same storage location. So, when we change the value of **x**, we are actually changing the value of **num1**.

11.6.3 Inline function

Normally the call statement to a function makes a compiler to jump to the functions (the definition of the functions are stored in STACKS) and also jump back to the instruction